10DaysOfDevOpsBasics

Day 1: Linux Fundamentals

- **Linux Introduction & Filesystem**
  - What is Linux and popular distributions
  - Filesystem Hierarchy Standard (/, /home, /etc, /var, /tmp, /opt, /usr, /bin)
  - Absolute vs relative paths
- **Basic Navigation & File Operations**
  - pwd, ls (with options -l -a -h -t), cd
  - mkdir –p, touch, cp -r, mv, rm -rf
  - cat, less, head, tail -f
- **Package Management**
  - apt update / apt upgrade / apt install / apt remove (Debian/Ubuntu)
  - yum install / yum update (RHEL/CentOS family)
  - dpkg -l vs apt list --installed
- **Scenario-Based Practice**
  - Scenario 1: You SSH into a production server and need to find the latest error logs under /var/log without knowing the exact filename.
  - Scenario 2: Create a temporary directory structure for a deployment, copy config files, safely, and verify contents before going live.
  - Scenario 3: A required tool isn't installed on the server - update package lists and install it using the correct package manager for the distro.

Day 2: Linux Users, Permissions & Processes

- **Users and Groups**
  - whoami, id, useradd, usermod, passwd, groupadd
  - chown, chgrp
  - sudo, visudo, /etc/sudoers basics
  - Difference between su and sudo
- **File Permissions**
  - Understanding rwx for owner/group/others

- chmod (numeric and symbolic)
- ls -l interpretation

➢ **Process Management**

- ps aux, top/htop
- kill, kill -9, jobs, bg, fg

➢ **Scenario-Based Practice**

- Scenario 1: A newly deployed app cannot write to its log directory – diagnose ownership and permissions and fix them correctly.
- Scenario 2: CPU is at 100% - identify the offending process, understand its parent, and terminate it safely.
- Scenario 3: Create a dedicated service user and group for an application and assign least-privilege permissions
- Scenario 4: Grant a new team member sudo access safely using visudo instead of editing sudoers directly.

Day 3: Linux Text Processing, Redirection & System Info

➢ **Text Editors**

- nano basics (save, exit)
- vim modes: insert vs command mode
- vim - :wq, :q!, dd, yy, p, :x

➢ **Text Search & Processing**

- grep -i -r, find (with -name -type -mtime)
- awk, sed, cut, sort, uniq, wc

➢ **I/O Redirection & Pipes**

- >>, <, 2>, &>, |, tee

➢ **System Information**

- df -h, du -sh, free -h, uname -a, uptime
- Basic log viewing with journalctl/tail

➢ **Scenario-Based Practice**

- Scenario 1: Extract all unique IP addresses that caused 500 errors from a 2GB access log

- Scenario 2: Disk is almost full – find the largest directories and files consuming space.
- Scenario 3: Pipe multiple commands together to generate a clean summary report of failed login attempts
- Scenario 4: SSH into a server with no GUI and edit a config file using vim - insert a line, save, and exit.

Day 4: Networking Fundamentals

- **Core Networking Concepts**
  - IP addressing basics: subnet masks, CIDR notation
  - Quick CIDR math: /24 = 254 usable hosts, /16 = larger network
  - TCP/IP basics, ports, sockets
  - Common ports (22, 80, 443, 3306, 5432, 8080)
  - DNS resolution flow
- **Essential Networking Commands**
  - ip a, ip r, ping, traceroute / mtr
  - ss -tulnp (or netstat), curl -l -v, wget, nc
- **DNS & Host Configuration**
  - dig, nslookup
  - /etc/hosts, /etc/resolv.conf
- **SSH Basics**
  - ssh-keygen, ssh-copy-id, passwordless authentication
  - scp, rsync
- **Scenario-Based Practice**
  - Scenario 1: Application cannot connect to the database – check listening ports, connectivity, and DNS resolution.
  - Scenario 2: Set up passwordless SSH between bastion host and application servers for automated deployments.
  - Scenario 3: A website is slow – use traceroute and curl to identify where latency is introduced.

Day 5: Shell Scripting Basics

- **Script Fundamentals**

- Shebang (#!/bin/bash), making scripts executable
- Variables, environment variables, quoting
- Positional parameters ($1 $2…), command substitution $()

➢ **Input & Conditionals**

- read command
- if-elif-else statements
- Test operators (-f -d -z -n, numeric & string comparisons)
- case statements

➢ **Exit Codes & Basic Error Checking**

- $?, simple success/failure checks

➢ **Scenario-Based Practice**

- Scenario 1: Write a script that accepts a filename as argument, checks if it exists and is readable, then prints its size
- Scenario 2: Create a script that verifies required environment variables are set before starting an application
- Scenario 3: Build a simple interactive menu script for common server checks (disk, memory, uptime)

Day 6: Shell Scripting Intermediate

➢ **Loops**

- for loops (lists, ranges, command output)
- while and until loops

➢ **Functions**

- Defining functions, passing arguments
- Local variables, return values

➢ **Automation Essentials**

- Basic error handling (set -e)
- Simple logging in scripts

➢ **Cron Jobs**

- crontab -e syntax

- Common scheduling patterns
- Environment differences in cron

➢ **Scenario-Based Practice**

- Scenario 1: Write a backup script that archives a directory with timestamp, logs success/failure, and keeps only last 7 days
- Scenario 2: Create a health-check script that loops through a list of services and restarts any that are down
- Scenario 3: Schedule the backup script with cron and verify it runs correctly with proper environment

Day 7: Git Fundamentals

➢ **Version Control Basics**

- What is Git (distributed model)
- Working directory → Staging area → Repository

➢ **Core Daily Commands**

- git init , git config (user.name user.email)
- git status , git add , git add -p
- git commit -m , git log --oneline --graph
- git diff , git restore / git checkout --

➢ **Scenario-Based Practice**

- Scenario 1: Initialize a repo for a shell script project, make meaningful atomic commits, and review history
- Scenario 2: Accidentally stage sensitive files - unstage them and add a proper .gitignore
- Scenario 3: Recover a deleted file from an earlier commit using git restore / checkout

Day 8: Git Branching & Collaboration

- **Branching & Merging**
  - git branch , git switch / git checkout -b
  - git merge
  - Resolving merge conflicts
- **Working with Remotes**
  - git remote -v , git clone
  - git push -u , git pull , git fetch
- **Safety & Collaboration Tools**
  - git stash , git stash pop
  - .gitignore
  - Basic GitHub/GitLab pull request flow
- **Scenario-Based Practice**
  - Scenario 1: Create a feature branch, make changes, push it, and open a pull request
  - Scenario 2: Two developers edit the same file - pull, resolve the merge conflict, and complete the merge
  - Scenario 3: You are mid-work and need to switch branches urgently - use stash safely and recover later

Day 9: Docker Fundamentals

- **Container Concepts**
  - Containers vs Virtual Machines
  - Images vs Containers
  - Docker architecture (client , daemon , registry)

- **Essential Container Commands**
    - docker pull , docker run (-d -p -e --name -it)
    - docker ps -a , docker stop / start / rm
    - docker logs -f , docker exec -it
- **Image Management**
    - docker images , docker rmi , docker tag
- **Scenario-Based Practice**
    - Scenario 1: Run an nginx container, map ports, and verify the website is accessible
    - Scenario 2: An application container keeps crashing - inspect logs and exec into it to diagnose
    - Scenario 3: Clean up stopped containers and unused images to free disk space on a busy host

Day 10: Docker Building & Orchestration Basics

- **Dockerfile & Custom Images**
    - Dockerfile instructions (FROM , RUN , COPY , WORKDIR , CMD , ENTRYPOINT , EXPOSE , ENV)
    - docker build -t
    - .dockerignore and basic layer caching
- **Data & Networking**
    - Volumes vs bind mounts
    - Basic Docker networks
- **Docker Compose Intro**
    - Simple docker-compose.yml structure
    - docker compose up / down / ps

- Multi-container service linking

- **Scenario-Based Practice**
  - Scenario 1: Write a Dockerfile for a simple web app, build the image, and run it with proper port mapping
  - Scenario 2: Add a volume so application data persists even after container removal
  - Scenario 3: Create a docker-compose.yml with a web app + database, start the stack, and verify they can communicate